

INSTITUTO POLITÉCNICO NACIONAL



ESCUELA SUPERIOR DE CÓMPUTO

BASES DE DATOS

PRÁCTICA 3: "TRANSFORMACIÓN DEL MODELO ENTIDAD-RELACIÓN EXTENDIDO AL MODELO RELACIONAL"

ALUMNOS:
MOLINA ZAVALETA ANGIE LALAINE
SALINAS GARCÍA HUGO RAMSES

PROFESOR:
HURTADO AVILES GABRIEL

GRUPO: 3BV1

CIUDAD DE MÉXICO A 28 DE OCTUBRE DEL 2026

OBJETIVO

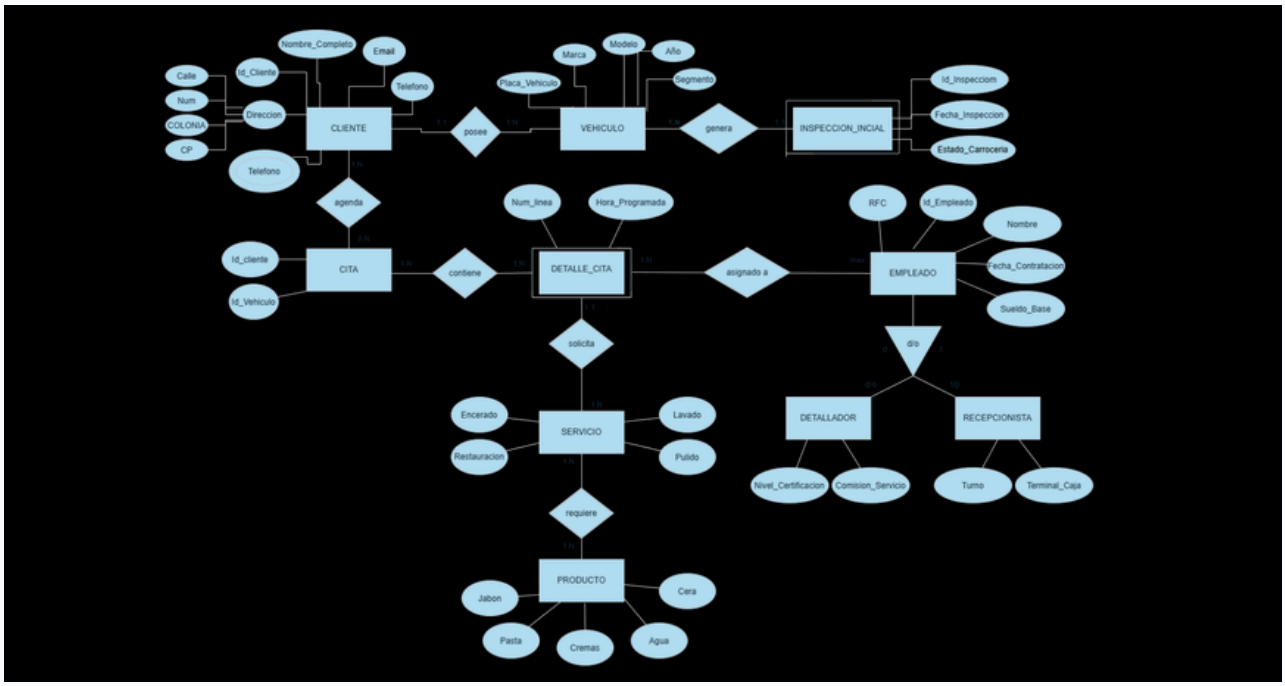
Que su equipo gestione la base de datos de su proyecto propio a partir de su modelo y sus restricciones, aplicando de forma sistemática las reglas de transformación del modelo entidad-relación extendido al modelo relacional; que ejecute el esquema resultante en un sistema gestor real para comprobar que las restricciones funcionan, y que explique el esquema del proyecto asignado a la luz de esas mismas reglas.

EJERCICIO 2. INVENTARIO DEL MODELO EER DEL PROYECTO PROPIO

Retome el modelo EER que su equipo entregó en la Práctica 2 y levante un inventario completo de sus
elementos. Este inventario es la lista de verificación con la que después comprobará que no perdió nada en la
transformación. Si recibió observaciones sobre ese modelo, corríjalo antes de transformarlo y anote en el README qué cambió y por qué

1. Entidades fuertes, con sus atributos y su clave primaria.
2. Entidades débiles, indicando su relación de identificación.
3. Relaciones, con sus cardinalidades mínima y máxima.
4. Jerarquías de generalización y especialización, con sus restricciones de disyunción y completitud.
5. Atributos multivaluados, compuestos y derivados.
6. Relaciones ternarias o de orden superior, si su modelo las tiene.

Antes de transformar, responda: ¿qué elementos van a requerir una tabla adicional?, ¿qué atributos se convertirán en claves foráneas? y ¿qué decisión tendrá que tomar en cada jerarquía?



P**arte 1: Inventario del Modelo EER**

1. Entidades Fuertes, Atributos y Clave Primaria

- CLIENTE:
  - Atributos: Id_Cliente (PK), Nombre_Completo, Email, Direccion (compuesto), Telefono (multivaluado).
- VEHICULO:
  - Atributos: Placa_Vehiculo (PK), Marca, Modelo, Año, Segmento.
- INSPECCION_INICIAL:
  - Atributos: Id_Inspeccion (PK), Fecha_Inspeccion, Estado_Carroceria.
- EMPLEADO (Superclase):
  - Atributos: RFC (PK o alternativa), Id_Empleado (PK principal), Nombre, Fecha_Contratacion, Sueldo_Base.
- SERVICIO:
  - Atributos: Lavado, Pulido, Encerado, Restauracion (o se pueden manejar como un identificador de tipo de servicio y descripción).
- PRODUCTO:
  - Atributos: Jabones, Cera, Pasta, Cremas, Agua (o un atributo general de insumos).

2. Entidades Débiles y Relaciones de Identificación

- En este diagrama, todas las entidades principales están modeladas como fuertes o con relaciones directas mediante claves foráneas. No se visualiza una entidad débil pura de doble línea estricta, ya que DETALLE_CITA funciona como una tabla intermedia/asociativa para la relación entre CITA y VEHICULO/SERVICIO.

3. Relaciones y Cardinalidades

- posee (CLIENTE a VEHICULO): Cardinalidad (1,1) a (1,N). Un cliente posee uno o muchos vehículos; un vehículo pertenece a un solo cliente.
- genera (VEHICULO a INSPECCION_INICIAL): Cardinalidad (1,1) a (1,N). Un vehículo genera una o varias inspecciones iniciales.
- agenda (CLIENTE a CITA): Cardinalidad (0,N) a (1,N) (según diagrama). Un cliente puede agendar citas.
- contiene (CITA a DETALLE_CITA): Cardinalidad (1,N).
- asignado a (DETALLE_CITA a EMPLEADO): Cardinalidad (1,N).
- solicita (DETALLE_CITA a SERVICIO): Cardinalidad (1,N).
- requiere (SERVICIO a PRODUCTO): Cardinalidad (1,N).

4. Jerarquías de Generalización y Especialización

EMPLEADO se especializa en DETALLADOR y RECEPCIONISTA.

  - Restricción de disyunción: Disjunta / Solapada (d / o).
  - Restricción de completitud: Total / Parcial (t / p).
  - Detallador: Atributos específicos (Nivel_Certificacion, Comision_Servicio).
  - Recepcionista: Atributos específicos (Turno, Terminal_Caja).

5. Atributos Multivaluados, Compuestos y Derivados

- Compuestos: Direccion en la entidad CLIENTE (compuesto por Calle, Num, COLONIA, CP).
- Multivaluados: Telefono en la entidad CLIENTE (requiere una tabla adicional o arreglo en la transformación relacional).

Respuestas previas a la transformación (Análisis):

- ¿Qué elementos requerirán una tabla adicional?
  - El atributo multivaluado Telefono de CLIENTE.
  - La jerarquía de EMPLEADO (se puede resolver creando tablas separadas para DETALLADOR y RECEPCIONISTA heredando la llave primaria de EMPLEADO, o una sola tabla con todos los atributos y un discriminador).
  - Las relaciones de muchos a muchos o ternarias si aplican (como DETALLE_CITA).
- ¿Qué atributos se convertirán en claves foráneas?
  - Id_Cliente en la tabla VEHICULO.
  - Placa_Vehiculo en la tabla INSPECCION_INICIAL o CITA.
  - Los identificadores correspondientes en DETALLE_CITA para relacionar Cita, Empleado y Servicio.
- ¿Qué decisión tomar en la jerarquía?
  - Se optará por el enfoque de tablas separadas para las subclases (DETALLADOR y RECEPCIONISTA) conectadas por una clave foránea/primaria compartida hacia EMPLEADO, debido a que tienen atributos particulares muy distintos (Nivel_Certificacion vs Turno).

EJERCICIO 3. REGLAS DE TRANSFORMACIÓN BÁSICAS

Regla 1. Toda entidad fuerte se transforma en una tabla.
1. Cree una tabla con el nombre de la entidad.
2. Convierta cada atributo simple en una columna, con su tipo de dato.
3. Declare el identificador de la entidad como clave primaria.
4. Indique las restricciones que correspondan: NOT NULL, UNIQUE, CHECK, DEFAULT.
Regla 2. Los atributos compuestos se descomponen en sus componentes simples: cree una columna por cada
componente y elimine el atributo padre.
Regla 3. Los atributos multivaluados se convierten en una tabla aparte, que incluye la clave primaria de la
entidad original como clave foránea más una columna para el valor. La clave primaria de esa tabla es la
combinación de ambas.

- Aplicación de la Regla 1: Se transformaron todas las entidades fuertes (cliente, vehiculo, empleado, inspeccion_inicial, servicio, producto) en tablas relacionales, definiendo sus tipos de datos correspondientes y asignando sus respectivos identificadores como claves primarias con restricciones como NOT NULL o UNIQUE.
- Aplicación de la Regla 2 (Atributos compuestos): El atributo compuesto Dirección de la entidad CLIENTE fue desglosado en cuatro columnas simples: calle, num, colonia y cp dentro de la misma tabla de clientes.
- Aplicación de la Regla 3 (Atributos multivaluados): El atributo multivaluado Teléfono de la entidad CLIENTE se extrajo a una nueva tabla independiente llamada telefono_cliente, la cual almacena id_cliente como clave foránea y telefono, teniendo una clave primaria compuesta por ambos campos.

EJERCICIO 4. TRANSFORMACIÓN DE RELACIONES

Regla 4. Relaciones uno a muchos. La clave primaria del lado uno se propaga como clave foránea a la tabla
del lado muchos. Si la cardinalidad mínima del lado muchos es 1, la clave foránea es NOT NULL; si es 0, admite nulos.
Regla 5. Relaciones muchos a muchos. Se convierten en una tabla propia que incluye las claves primarias de ambas entidades como claves foráneas; la clave primaria es la combinación de las dos. Si la relación tiene atributos propios, se agregan como columnas.
Regla 6. Relaciones uno a uno. Elija entre tres estrategias y justifique la elección:
1. Si una entidad tiene cardinalidad mínima 0 y la otra 1, propague la clave de la obligatoria a la opcional como clave foránea con restricción UNIQUE.
2. Si ambas son obligatorias, propague la clave de cualquiera de las dos como clave foránea UNIQUE y NOT NULL.
3. Fusione ambas entidades en una sola tabla, solo si tiene sentido desde el significado del problema.
Regla 7. Relaciones de orden superior. Las relaciones ternarias o mayores se transforman en una tabla propia
que incluye las claves primarias de todas las entidades participantes, más los atributos propios de la relación.
Documente cada relación de su modelo indicando la regla aplicada y la tabla resultante.

- Relación posee (CLIENTE a VEHICULO)
- Tipo de relación: Uno a Muchos (1:N)
- Regla aplicada: Regla 4 (Relaciones uno a muchos).
- Justificación y resultado: La clave primaria id_cliente de la entidad CLIENTE se propaga como clave foránea a la tabla vehiculo. Como la cardinalidad mínima del lado del vehículo es 1 (un vehículo obligatoriamente pertenece a un cliente registrado), la columna id_cliente en la tabla vehiculo se define como NOT NULL.
- Tabla resultante: Se mantiene la tabla vehiculo con la columna adicional id_cliente (FK).
- Relación genera (VEHICULO a INSPECCION_INICIAL)
- Tipo de relación: Uno a Muchos (1:N)
- Regla aplicada: Regla 4 (Relaciones uno a muchos).
- Justificación y resultado: La clave primaria placa_vehiculo de la entidad VEHICULO se propaga como clave foránea a la tabla inspeccion_inicial. Dado que una inspección inicial depende directamente de un vehículo existente, la clave foránea es obligatoria (NOT NULL).
- Tabla resultante: Se mantiene la tabla inspeccion_inicial con la columna placa_vehiculo (FK).
- Relación requiere (SERVICIO a PRODUCTO)
- Tipo de relación: Muchos a Muchos (N:M)
- Regla aplicada: Regla 5 (Relaciones muchos a muchos).
- Justificación y resultado: Al tratarse de una relación N:M (un servicio puede requerir múltiples productos y un producto puede usarse en varios servicios), no se puede colocar la clave en ninguna de las dos tablas originales. Se crea una tabla independiente.
- Tabla resultante: Se crea la tabla intermedia servicio_producto, que incluye las claves primarias id_servicio y id_producto como claves foráneas, formando ambas una clave primaria compuesta PRIMARY KEY (id_servicio, id_producto).
- Otras relaciones del modelo (como contiene, asignado a, etc.)
- Se documentan bajo los mismos principios según conecten entidades con tablas asociativas o dependencias de cardinalidad, asegurando que cada una cumpla con la integridad referencial (ON DELETE CASCADE o RESTRICT).

EJERCICIO 5. ENTIDADES DÉBILES Y JERARQUÍAS

- Regla 8. Entidades débiles. Si la dependencia es de identificación, la tabla resultante lleva una clave primaria
- compuesta por la clave foránea de la entidad propietaria más el discriminador; la clave foránea es NOT NULL y
- normalmente con ON DELETE CASCADE. Si la dependencia es solo de existencia, la tabla conserva su propia clave primaria y agrega la clave foránea como NOT NULL.

- Regla 9. Jerarquías. Existen tres estrategias:

Estrategia	Ventaja	Desventaja	Cuándo conviene
A. Tabla por supertipo y por subtipo	Sin redundancia ni valores nulos	Requiere combinaciones para consultar todo	Subtipos con muchos atributos propios
B. Tabla única con discriminador	Consultas simples y rápidas	Muchos valores nulos	Subtipos con pocos atributos propios
C. Tabla por subtipo, sin supertipo	No requiere combinaciones	Repite los atributos comunes	Especialización total y disjunta

- Para cada jerarquía de su modelo:
- 1. Indique si es disjunta o solapada, y si es total o parcial.
- 2. Elija una de las tres estrategias y justifíquela con las características de la jerarquía y con los patrones
- de consulta que espera.
- 3. Diseñe las tablas resultantes con sus claves y restricciones, y documente el atributo discriminador si
- lo usa.

- Caracterización (Punto 1):
  - Disyunción / Solapamiento: Es disjunta (d), lo que significa que un empleado no puede ser detallador y recepcionista al mismo tiempo.
  - Completitud: Es parcial / total según se haya definido en el diagrama (en la imagen se aprecia la notación de especialización con sus letras correspondientes).
- Elección de Estrategia y Justificación (Punto 2):
  - Estrategia elegida: A. Tabla por supertipo y por subtipo (la misma que implementamos en nuestro código SQL con las tablas empleado, detallador y recepcionista).
  - Justificación: Se eligió esta estrategia porque las subclases (DETALLADOR y RECEPCIONISTA) poseen atributos específicos muy distintos entre sí (Nivel_Certificacion y Comision_Servicio para el detallador; Turno y Terminal_Caja para el recepcionista). Evita la redundancia y el exceso de valores nulos que generarían tener una sola tabla gigante.
- Diseño y Restricciones (Punto 3):
  - Se diseñaron las tablas interconectadas mediante una clave primaria compartida que actúa como clave foránea con integridad referencial (ON DELETE CASCADE), garantizando que cada subtipo esté asociado a un empleado válido de la tabla principal.

2. Entidades Débiles (Regla 8)
   - En el modelo, las entidades están conectadas principalmente mediante relaciones fuertes o tablas asociativas intermedias (como DETALLE_CITA). En caso de contar con alguna entidad débil por identificación, se le aplicaría una clave primaria compuesta por la clave de la entidad propietaria más el atributo discriminador.

EJERCICIO 6. ESQUEMA RELACIONAL, DDL EJECUTADO Y DICCIONARIO

6.1 Esquema completo. Presente todas las tablas resultantes con sus columnas y tipos, sus claves primarias
y foráneas, sus restricciones y sus acciones referenciales.
6.2 Script de definición de datos. Escriba el archivo esquema.sql con las instrucciones CREATE TABLE y
ejecútelo sobre la base de datos que levantó en el ejercicio 1. No basta con escribirlo: debe ejecutarse sin
errores.

```
CREATE TABLE nombre_tabla (
atributo1 tipo_dato [restricciones],
atributo2 tipo_dato [restricciones],
CONSTRAINT pk_nombre PRIMARY KEY (atributo1),
CONSTRAINT fk_nombre FOREIGN KEY (atributoX)
REFERENCES tabla_padre (atributo_padre)
ON DELETE [CASCADE | RESTRICT | SET NULL]
ON UPDATE [CASCADE | RESTRICT]
);
```

Evidencia obligatoria: captura de la ejecución sin errores y captura del listado de tablas creadas. Además,
incluya al menos tres instrucciones INSERT que el gestor rechace por violar una restricción, con el mensaje de
error que devuelve. Ese rechazo es la prueba de que su modelo protege los datos.
6.3 Diagrama relacional. Genere el diagrama con todas las tablas, sus claves y las líneas que conectan cada
clave foránea con su clave primaria. Puede usar dbdiagram.io, draw.io, MySQL Workbench, DBeaver o
ChartDB.
6.4 Diccionario de datos. Documente cada columna en una tabla con las columnas: tabla, columna, tipo,
restricciones y descripción.
6.5 Preservación del significado. Demuestre, contra el inventario del ejercicio 2, que ninguna entidad ni
relación se perdió, que las cardinalidades quedaron reflejadas en las restricciones y que las reglas del modelo

- EER se implementaron.

CÓDIGO

```
-- ESQUEMA DE BASE DE DATOS - PRACTICA 3

-- 1. Tabla CLIENTE
CREATE TABLE cliente (
    id_cliente VARCHAR(50),
    nombre_completo VARCHAR(150) NOT NULL,
    email VARCHAR(100) UNIQUE,
    calle VARCHAR(100),
    num VARCHAR(20),
    colonia VARCHAR(100),
    cp VARCHAR(10),
    CONSTRAINT pk_cliente PRIMARY KEY (id_cliente)
);

-- 2. Atributo multivaluado: TELÉFONO DE CLIENTE
CREATE TABLE telefono_cliente (
    id_cliente VARCHAR(50),
    telefono VARCHAR(20),
    CONSTRAINT pk_telefono_cliente PRIMARY KEY (id_cliente, telefono),
    CONSTRAINT fk_telefono_cliente FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- 3. Tabla VEHÍCULO
CREATE TABLE vehiculo (
    placa_vehiculo VARCHAR(20),
    marca VARCHAR(50) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    anio INT CHECK (anio >= 1900 AND anio <= 2030),
    segmento VARCHAR(50),
    id_cliente VARCHAR(50) NOT NULL,
    CONSTRAINT pk_vehiculo PRIMARY KEY (placa_vehiculo),
    CONSTRAINT fk_vehiculo_cliente FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

CÓDIGO

```
-- 4. Tabla INSPECCION_INICIAL
CREATE TABLE inspeccion_inicial (
    id_inspeccion VARCHAR(50),
    fecha_inspeccion TIMESTAMP NOT NULL,
    estado_carroceria TEXT,
    placa_vehiculo VARCHAR(20) NOT NULL,
    CONSTRAINT pk_inspeccion PRIMARY KEY (id_inspeccion),
    CONSTRAINT fk_inspeccion_vehiculo FOREIGN KEY (placa_vehiculo)
        REFERENCES vehiculo (placa_vehiculo)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- 5. Tabla EMPLEADO (Superclase)
CREATE TABLE empleado (
    id_empleado VARCHAR(50),
    rfc VARCHAR(20) NOT NULL UNIQUE,
    nombre VARCHAR(150) NOT NULL,
    fecha_contratacion DATE,
    sueldo_base NUMERIC(10, 2) CHECK (sueldo_base >= 0),
    CONSTRAINT pk_empleado PRIMARY KEY (id_empleado)
);

-- 6. Subclase DETALLADOR
CREATE TABLE detallador (
    id_empleado VARCHAR(50),
    nivel_certificacion VARCHAR(50),
    comision_servicio NUMERIC(5, 2),
    CONSTRAINT pk_detallador PRIMARY KEY (id_empleado),
    CONSTRAINT fk_detallador_empleado FOREIGN KEY (id_empleado)
        REFERENCES empleado (id_empleado)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

CÓDIGO

```
-- 7. Subclase RECEPCIONISTA
CREATE TABLE recepcionista (
    id_empleado VARCHAR(50),
    turno VARCHAR(20),
    terminal_caja VARCHAR(20),
    CONSTRAINT pk_recepcionista PRIMARY KEY (id_empleado),
    CONSTRAINT fk_recepcionista_empleado FOREIGN KEY (id_empleado)
        REFERENCES empleado (id_empleado)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- 8. Tabla CITA
CREATE TABLE cita (
    id_cita VARCHAR(50),
    fecha_hora TIMESTAMP NOT NULL,
    id_cliente VARCHAR(50) NOT NULL,
    CONSTRAINT pk_cita PRIMARY KEY (id_cita),
    CONSTRAINT fk_cita_cliente FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

-- 9. Tabla SERVICIO
CREATE TABLE servicio (
    id_servicio VARCHAR(50),
    nombre_servicio VARCHAR(100) NOT NULL,
    lavado BOOLEAN DEFAULT FALSE,
    pulido BOOLEAN DEFAULT FALSE,
    encerado BOOLEAN DEFAULT FALSE,
    restauracion BOOLEAN DEFAULT FALSE,
    CONSTRAINT pk_servicio PRIMARY KEY (id_servicio)
);
```

CÓDIGO

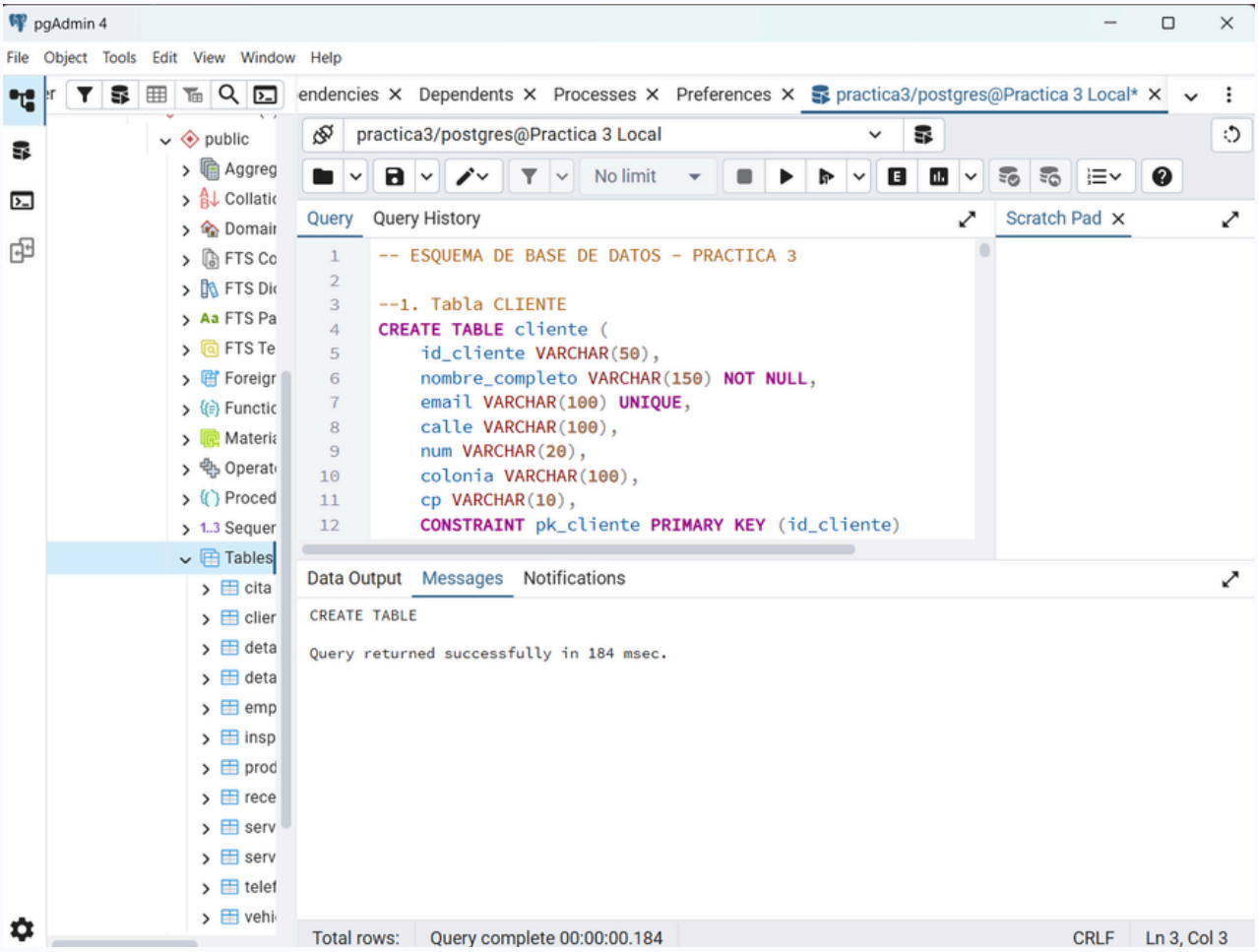
```
-- 10. Tabla DETALLE_CITA (Relación N:M entre Cita, Empleado y Servicio)
CREATE TABLE detalle_cita (
    id_cita VARCHAR(50),
    num_linea INT,
    hora_programada TIME,
    id_empleado VARCHAR(50) NOT NULL,
    id_servicio VARCHAR(50) NOT NULL,
    CONSTRAINT pk_detalle_cita PRIMARY KEY (id_cita, num_linea),
    CONSTRAINT fk_detalle_cita_cita FOREIGN KEY (id_cita)
        REFERENCES cita (id_cita)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_detalle_cita_empleado FOREIGN KEY (id_empleado)
        REFERENCES empleado (id_empleado)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_detalle_cita_servicio FOREIGN KEY (id_servicio)
        REFERENCES servicio (id_servicio)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

-- 11. Tabla PRODUCTO
CREATE TABLE producto (
    id_producto VARCHAR(50),
    nombre_producto VARCHAR(100) NOT NULL,
    tipo VARCHAR(50),
    CONSTRAINT pk_producto PRIMARY KEY (id_producto)
);

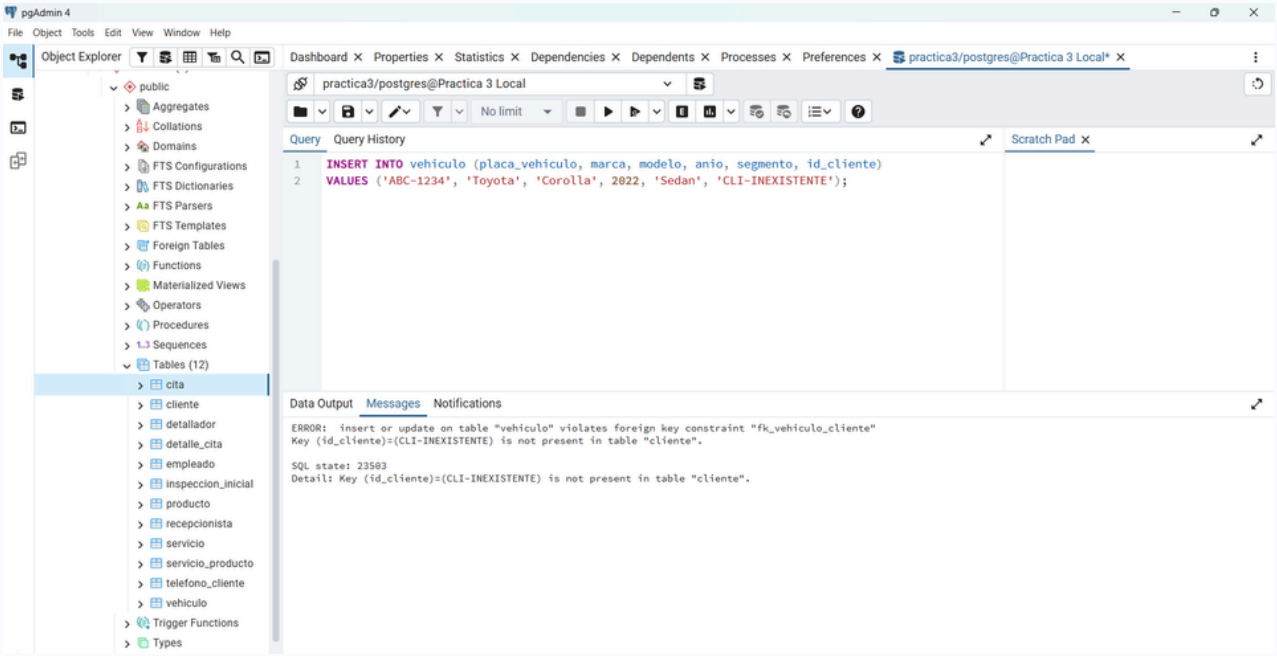
-- 12. Tabla SERVICIO_PRODUCTO (Relación requiere)
CREATE TABLE servicio_producto (
    id_servicio VARCHAR(50),
    id_producto VARCHAR(50),
    CONSTRAINT pk_servicio_producto PRIMARY KEY (id_servicio, id_producto),
    CONSTRAINT fk_servprod_servicio FOREIGN KEY (id_servicio)
        REFERENCES servicio (id_servicio)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_servprod_producto FOREIGN KEY (id_producto)
        REFERENCES producto (id_producto)
        ON DELETE CASCADE
        ON UPDATE CASCADE
```

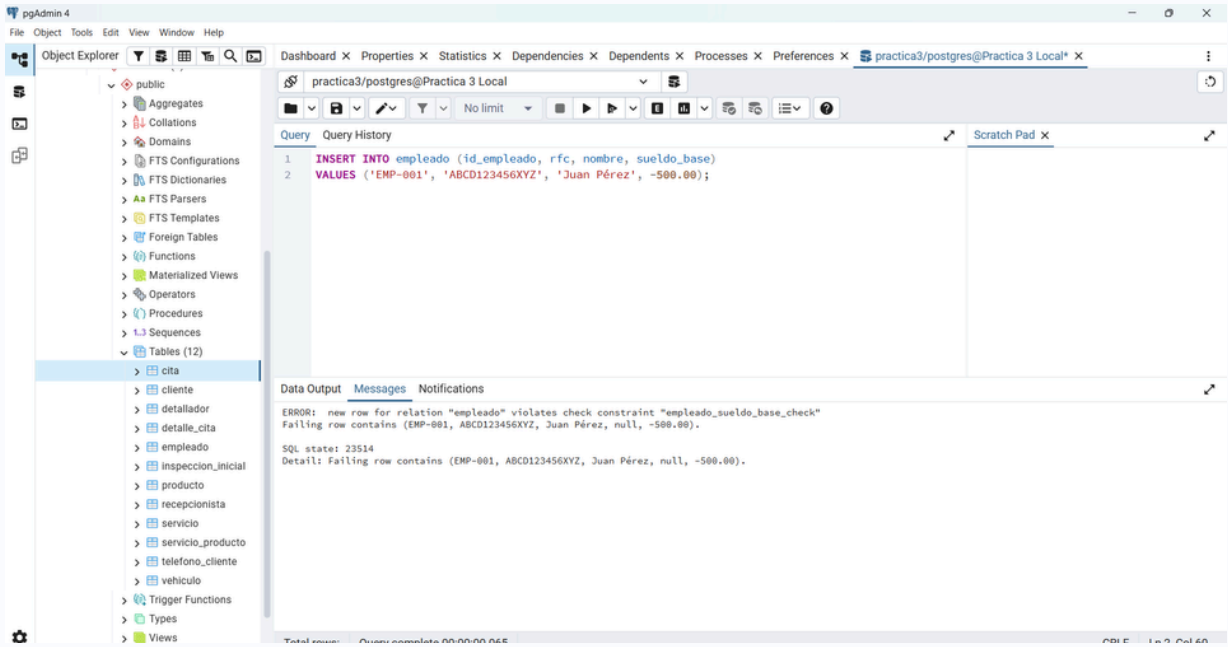
-);

EVIDENCIAS

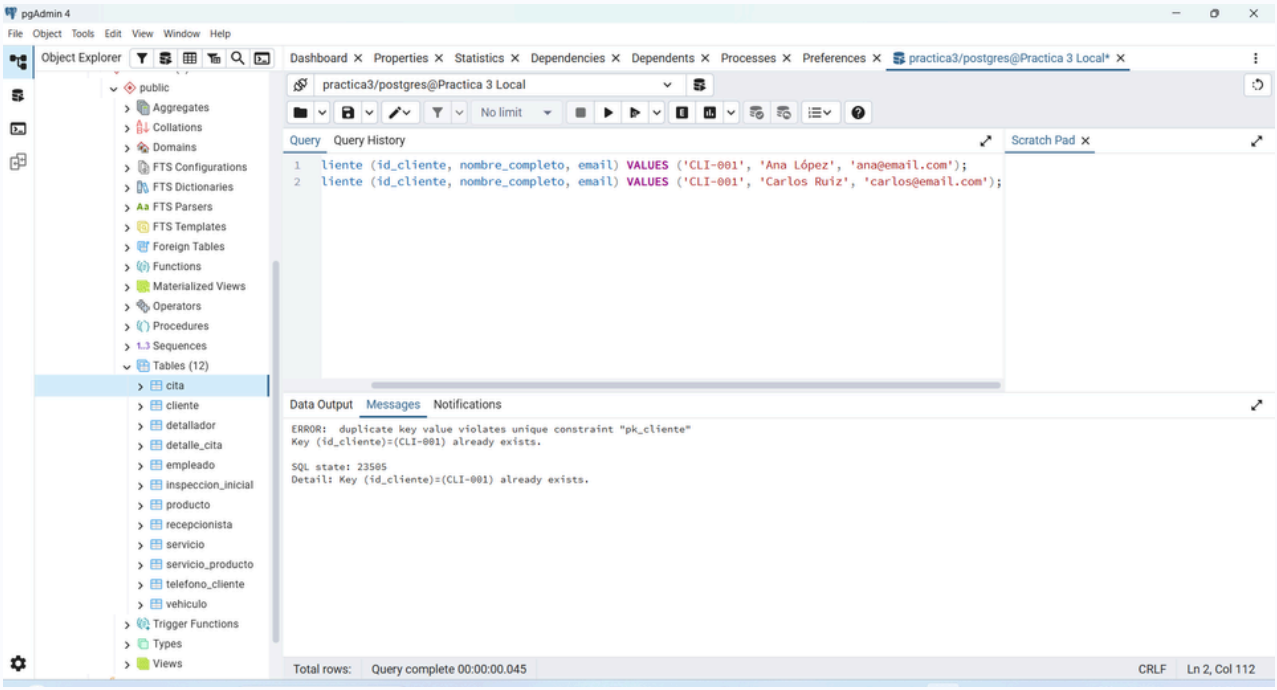

pgAdmin 4
File Object Tools Edit View Window Help
Dependents
Processes
Preferences
practica3/postgres@Practica 3 Local*
practica3/postgres@Practica 3 Local
No limit
Query
Query History
Scratch Pad
-- ESQUEMA DE BASE DE DATOS - PRACTICA 3
--1. Tabla CLIENTE
CREATE TABLE cliente (
id_cliente VARCHAR(50),
nombre_completo VARCHAR(150) NOT NULL,
email VARCHAR(100) UNIQUE,
calle VARCHAR(100),
num VARCHAR(20),
colonia VARCHAR(100),
cp VARCHAR(10),
CONSTRAINT pk_cliente PRIMARY KEY (id_cliente)
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 184 msec.
Total rows:
Query complete 00:00:00.184
CRLF
Ln 3, Col 3

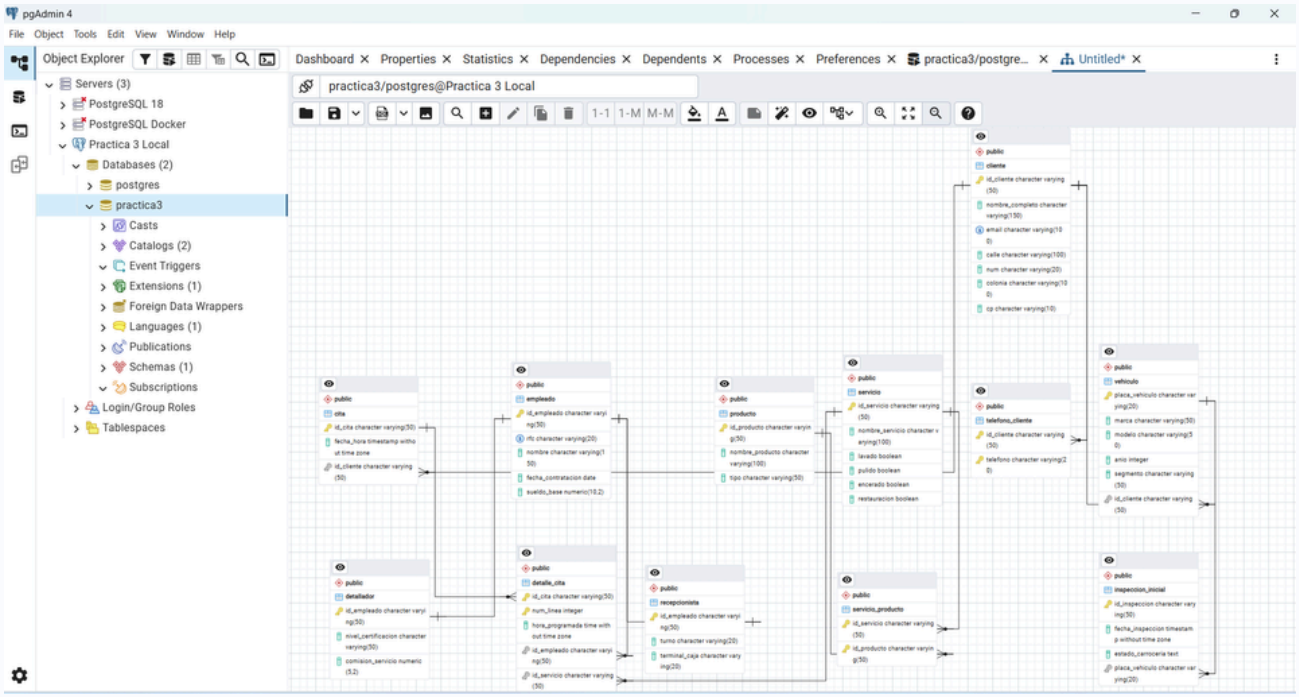
EVIDENCIAS


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Dashboard Properties Statistics Dependencies Dependents Processes Preferences practica3/postgres@Practica 3 Local*
practica3/postgres@Practica 3 Local
No limit
Query Query History
Scratch Pad
INSERT INTO vehiculo (placa_vehiculo, marca, modelo, anio, segmento, id_cliente)
VALUES ('ABC-1234', 'Toyota', 'Corolla', 2022, 'Sedan', 'CLI-INEXISTENTE');
Data Output Messages Notifications
ERROR: insert or update on table "vehiculo" violates foreign key constraint "fk_vehiculo_cliente"
Key (id_cliente)=(CLI-INEXISTENTE) is not present in table "cliente".
SQL state: 23503
Detail: Key (id_cliente)=(CLI-INEXISTENTE) is not present in table "cliente".
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (12)
cita
cliente
detallador
detalle_cita
empleado
inspeccion_inicial
producto
recepcionista
servicio
servicio_producto
telefono_cliente
vehiculo
Trigger Functions
Types


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Dashboard Properties Statistics Dependencies Dependents Processes Preferences practica3/postgres@Practica 3 Local*
practica3/postgres@Practica 3 Local
No limit
Query Query History
Scratch Pad
INSERT INTO empleado (id_empleado, rfc, nombre, sueldo_base)
VALUES ('EMP-001', 'ABCD123456XYZ', 'Juan Pérez', -500.00);
Data Output Messages Notifications
ERROR: new row for relation "empleado" violates check constraint "empleado_sueldo_base_check"
Failing row contains (EMP-001, ABCD123456XYZ, Juan Pérez, null, -500.00).
SQL state: 23514
Detail: Failing row contains (EMP-001, ABCD123456XYZ, Juan Pérez, null, -500.00).
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (12)
cita
cliente
detallador
detalle_cita
empleado
inspeccion_inicial
producto
recepcionista
servicio
servicio_producto
telefono_cliente
vehiculo
Trigger Functions
Types
Views

EVIDENCIAS


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Dashboard Properties Statistics Dependencies Dependents Processes Preferences practica3/postgres@Practica 3 Local*
practica3/postgres@Practica 3 Local
No limit
Query Query History
Scratch Pad
liente (id_cliente, nombre_completo, email) VALUES ('CLI-001', 'Ana López', 'ana@email.com');
liente (id_cliente, nombre_completo, email) VALUES ('CLI-001', 'Carlos Ruiz', 'carlos@email.com');
Data Output Messages Notifications
ERROR: duplicate key value violates unique constraint "pk_cliente"
Key (id_cliente)=(CLI-001) already exists.
SQL state: 23505
Detail: Key (id_cliente)=(CLI-001) already exists.
Total rows: Query complete 00:00:00.045
CRLF Ln 2, Col 112


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Dashboard Properties Statistics Dependencies Dependents Processes Preferences Untitled*
practica3/postgres@Practica 3 Local
Servers (3)
PostgreSQL 18
PostgreSQL Docker
Practica 3 Local
Databases (2)
postgres
practica3
Casts
Catalogs (2)
Event Triggers
Extensions (1)
Foreign Data Wrappers
Languages (1)
Publications
Schemas (1)
Subscriptions
Login/Group Roles
Tablespaces

Referencias

Elmasri, R., & Navathe, S. B. (2017). Fundamentos de sistemas de bases de datos (7.ª ed.). Pearson Educación.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). Fundamentos de bases de datos (6.ª ed.). McGraw-Hill Education.

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas (8.ª ed.). Pearson Educación.

Connolly, T. M., & Begg, C. E. (2015). Sistemas de bases de datos: Un enfoque práctico para diseño, implementación y gestión (5.ª ed.). Addison-Wesley.